



Universidad
de Concepción

FACULTAD DE
CIENCIAS
BIOLÓGICAS
Universidad de Concepción



Pre-Processing



Quality analysis and global statistics

Due to different source errors that might be present in our sequences, it is crucial to perform a quality check **BEFORE** we start the analysis.

The gold standard program to do quality checks is **FASTQC**



Babraham Bioinformatics

[About](#) | [People](#) | [Services](#) | [Projects](#) | [Training](#) | [Publications](#)



FastQC

Function	A quality control tool for high throughput sequence data.
Language	Java
Requirements	A suitable Java Runtime Environment The Picard BAM/SAM Libraries (included in download)
Code Maturity	Stable. Mature code, but feedback is appreciated.
Code Released	Yes, under GPL v3 or later .
Initial Contact	Simon Andrews
Download Now	

<https://www.bioinformatics.babraham.ac.uk/projects/fastqc/>

FASTQC Quality analysis

- > **Global statistics** (Number of sequences, read length, %GC)
- > **Phred quality score** (Per base and global Q-score)
- > **%GC** (most important for genome assembly)
- > **Undetermined Base** (% N bases along the reads)
- > **Phred quality score** (Per base and global Q-score)
- > **Distribution of read length**
- > **Read Duplication**
- > **Overrepresented sequences**
- > **Adapter Content**

FASTQC Report

Good Quality Run

Summary

- ✓ [Basic Statistics](#)
- ✓ [Per base sequence quality](#)
- ✓ [Per tile sequence quality](#)
- ✓ [Per sequence quality scores](#)
- ✓ [Per base sequence content](#)
- ✓ [Per sequence GC content](#)
- ✓ [Per base N content](#)
- ✓ [Sequence Length Distribution](#)
- ✓ [Sequence Duplication Levels](#)
- ✓ [Overrepresented sequences](#)
- ✓ [Adapter Content](#)



OK



warning



Fail

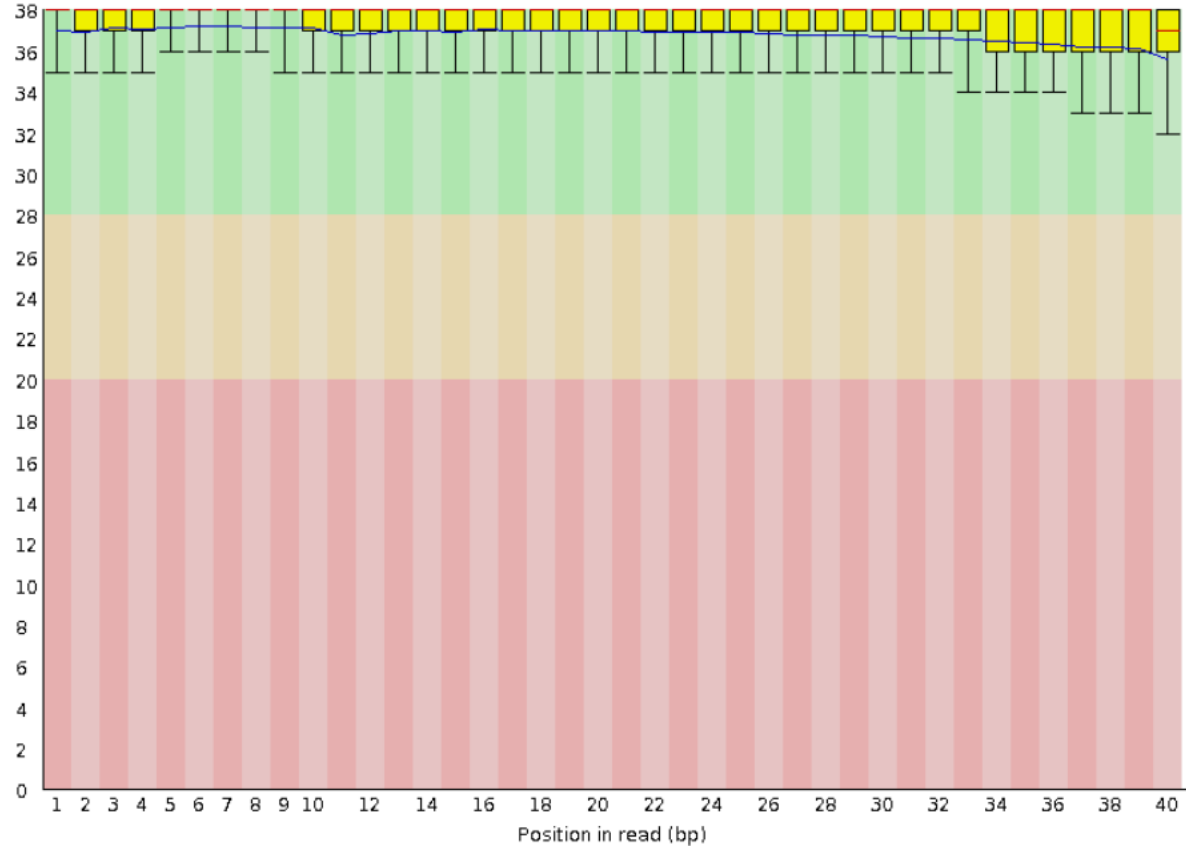
Bad Quality Run

Summary

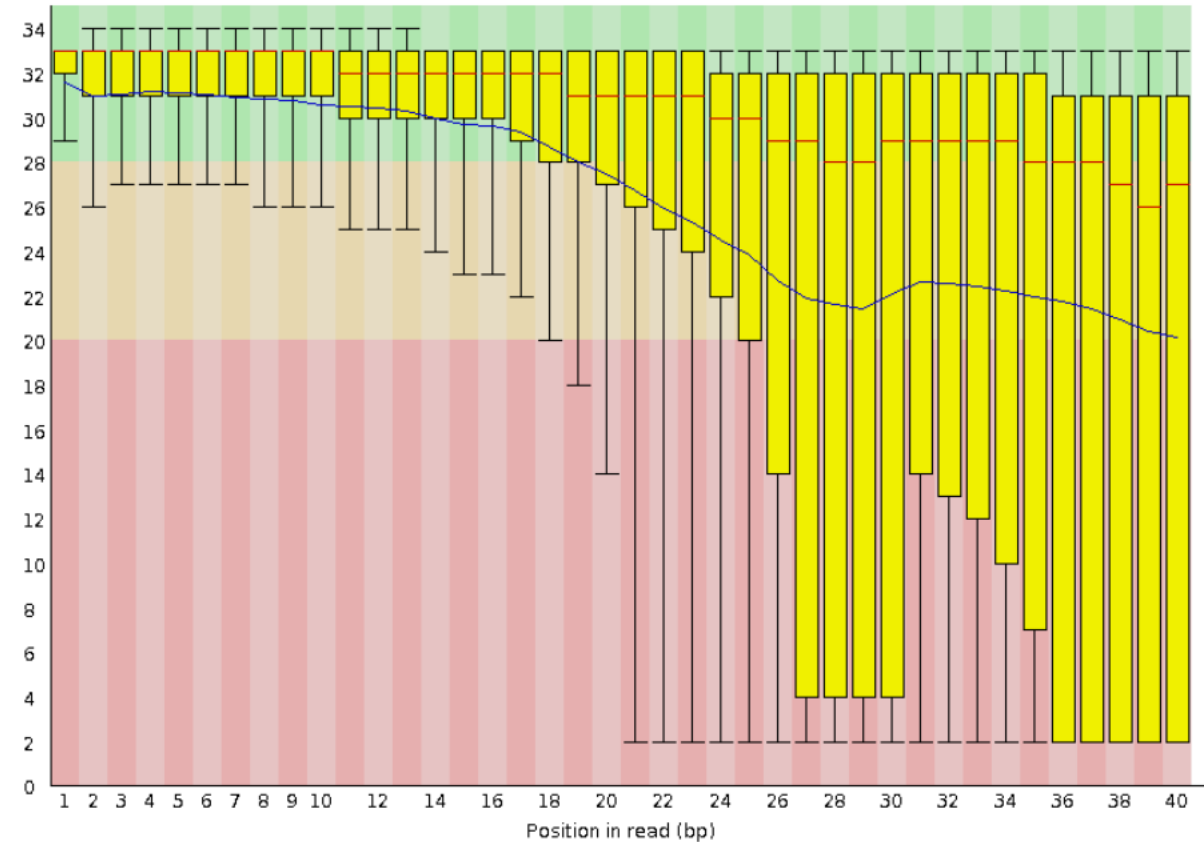
- ✓ [Basic Statistics](#)
- ✗ [Per base sequence quality](#)
- ✗ [Per tile sequence quality](#)
- ✓ [Per sequence quality scores](#)
- ! [Per base sequence content](#)
- ! [Per sequence GC content](#)
- ✓ [Per base N content](#)
- ✓ [Sequence Length Distribution](#)
- ! [Sequence Duplication Levels](#)
- ! [Overrepresented sequences](#)
- ✓ [Adapter Content](#)

Per Base quality

Good Quality Run



Bad Quality Run



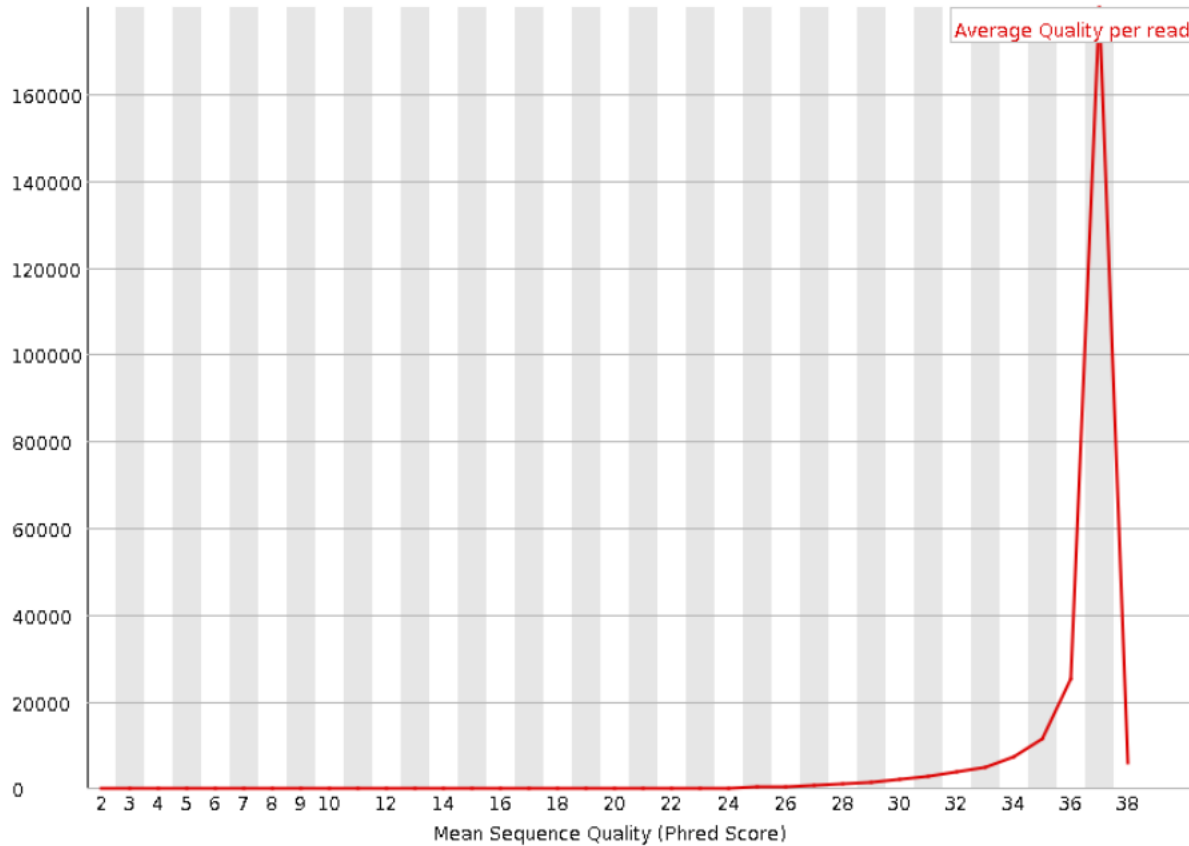
— median value
— mean value
inter-quartile range (25-75%)
whiskers 10% and 90% of points

How can I Improve?:

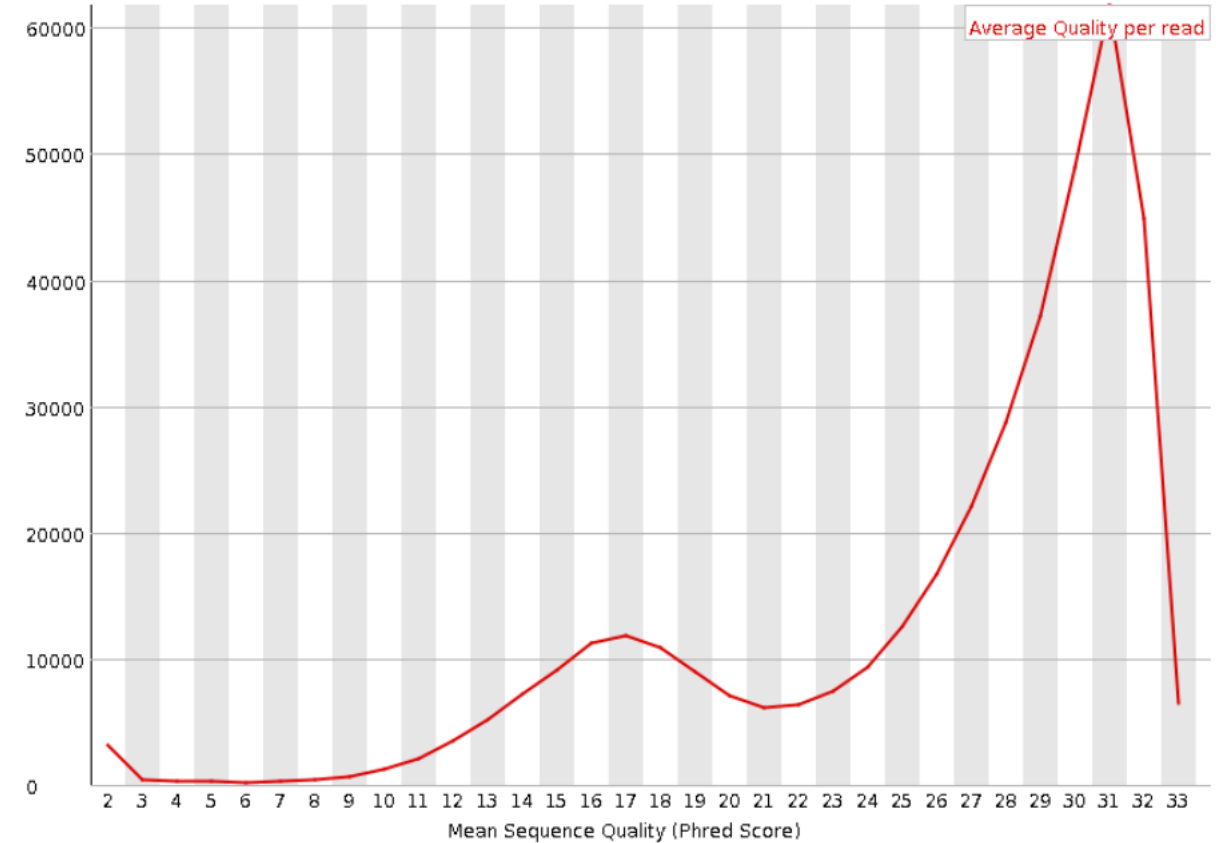
Trim bad quality bases from 3' (<20-25 Phred score)
Do not keep reads shorter than 35bp

Per sequence quality value

Good Quality Run



Bad Quality Run

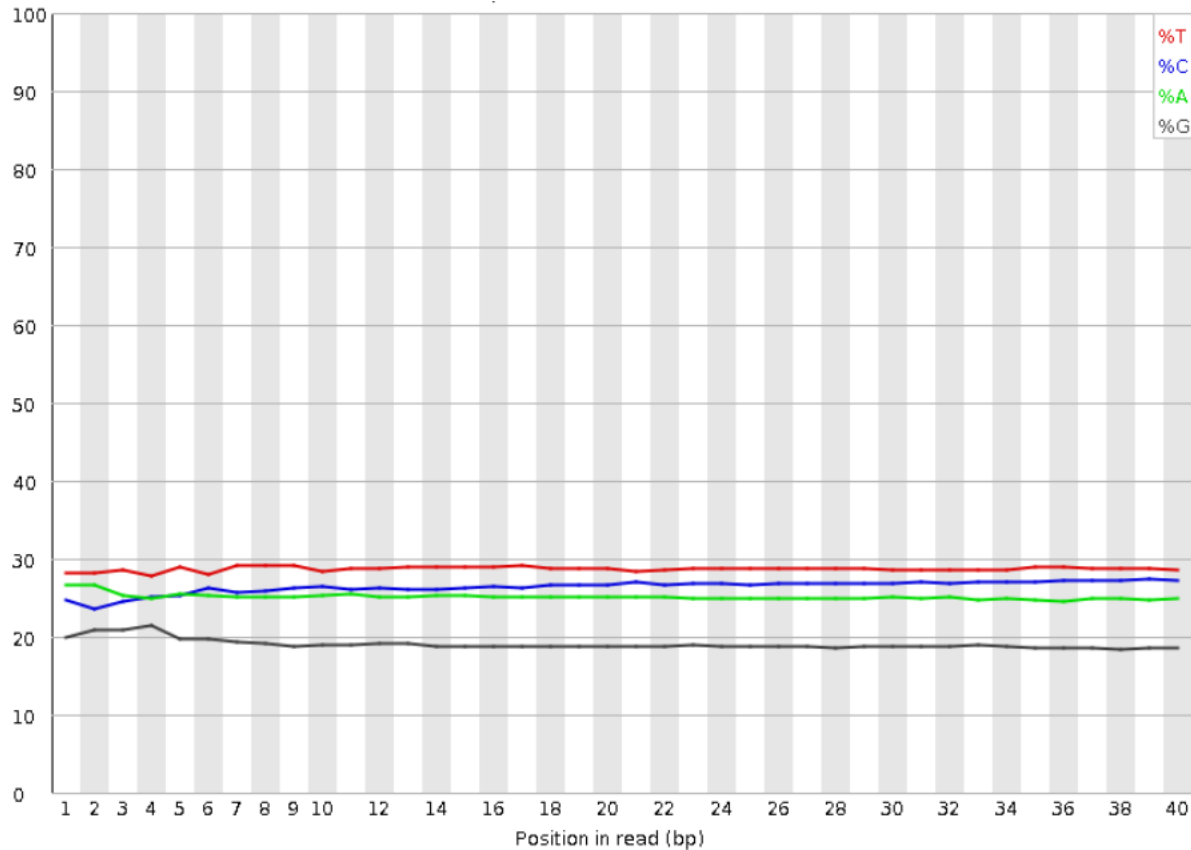


How can I Improve?:

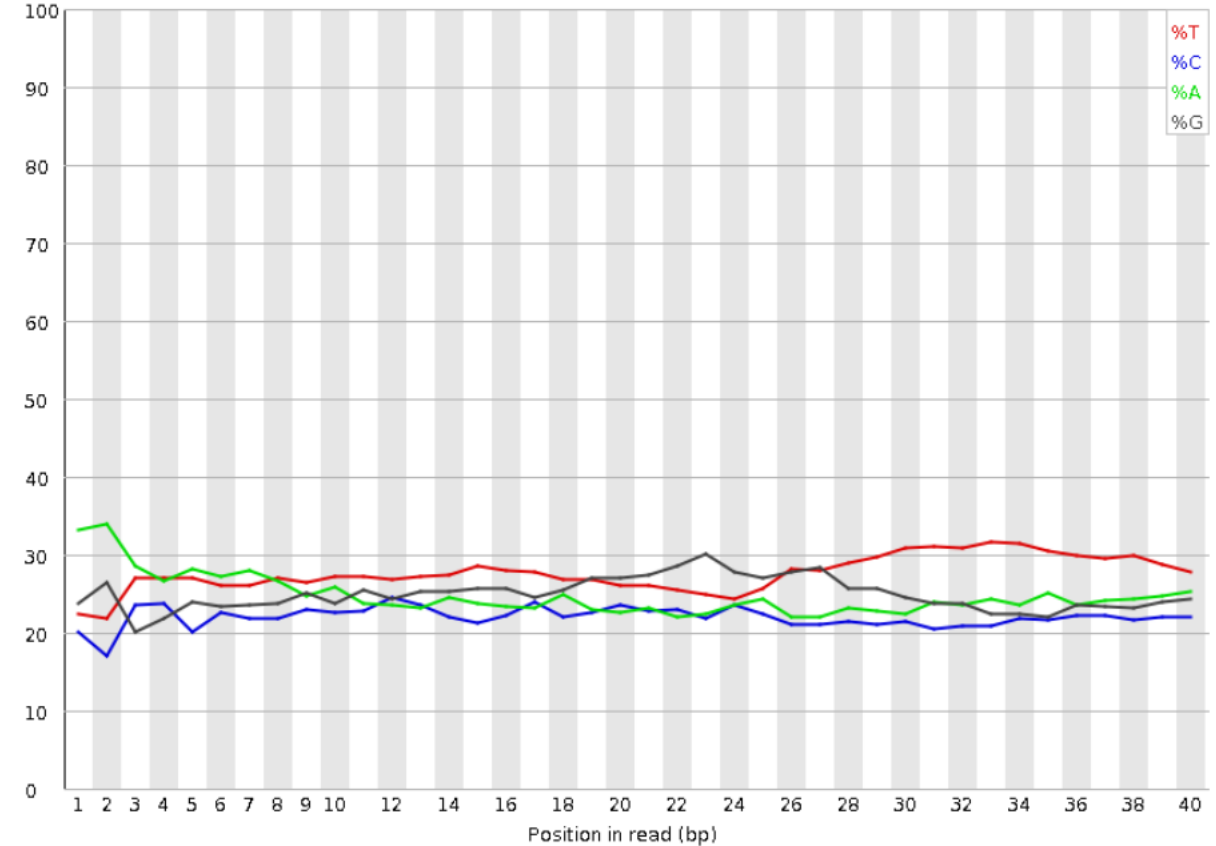
Remove reads with average phred score smaller than 20

Base content

Good Quality Run



Bad Quality Run

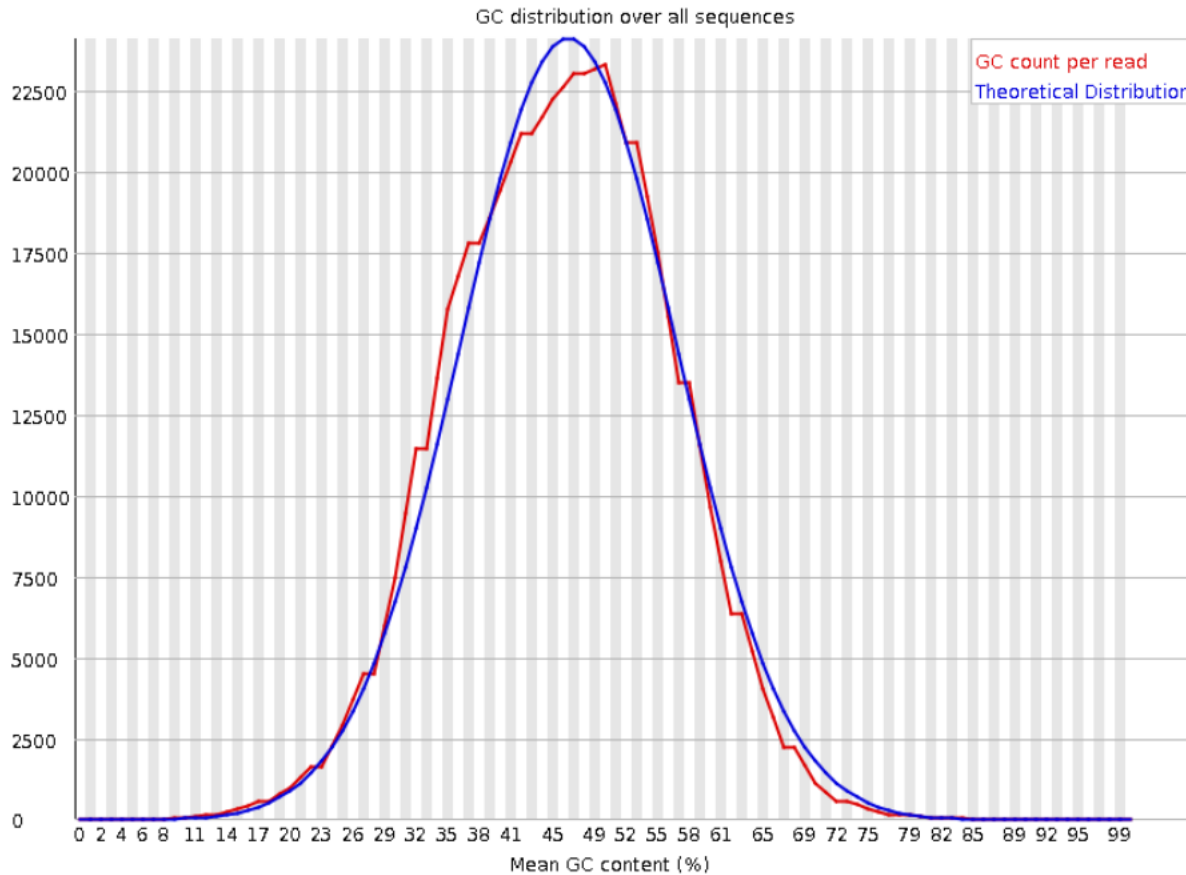


How can I Improve?:

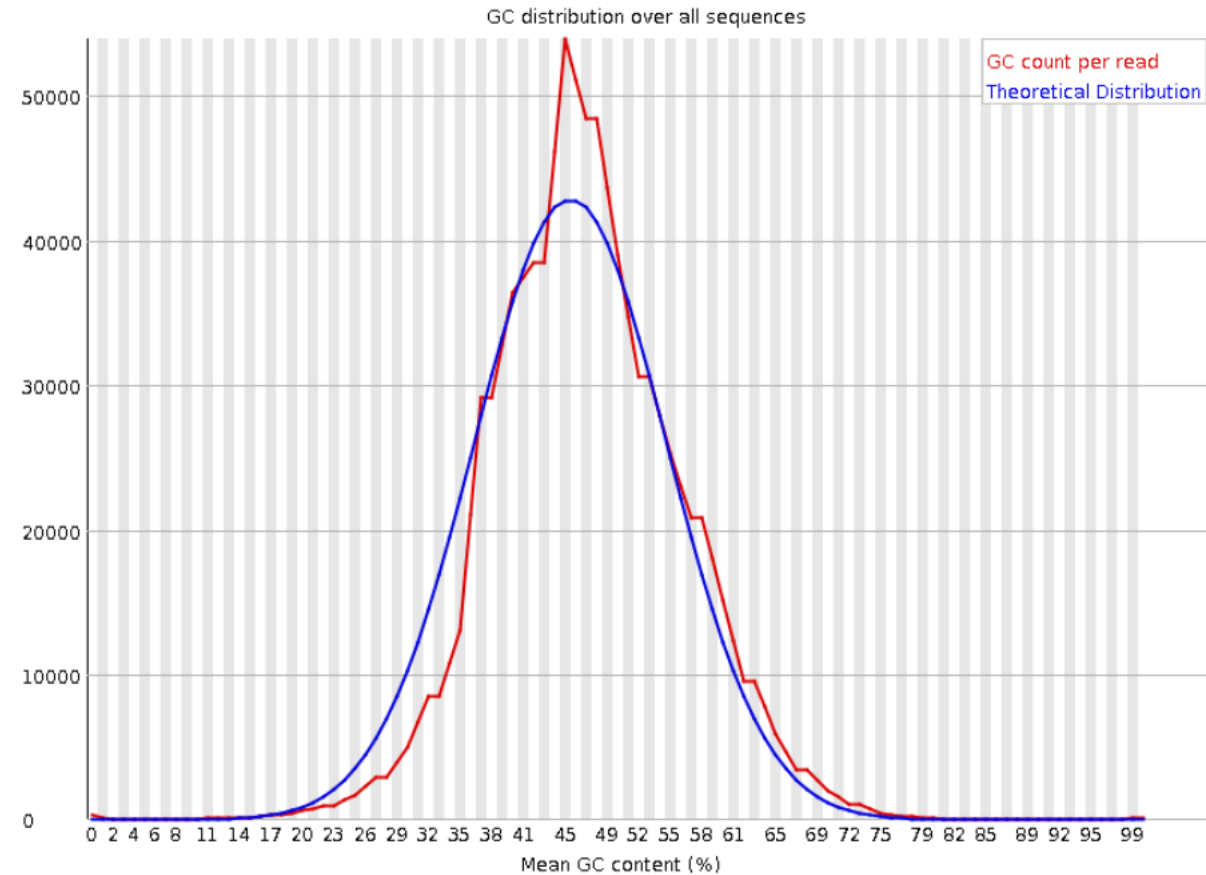
If base bias is present at the ends of the read, we can cut bases from the end and start of the read.

GC content

Good Quality Run



Bad Quality Run

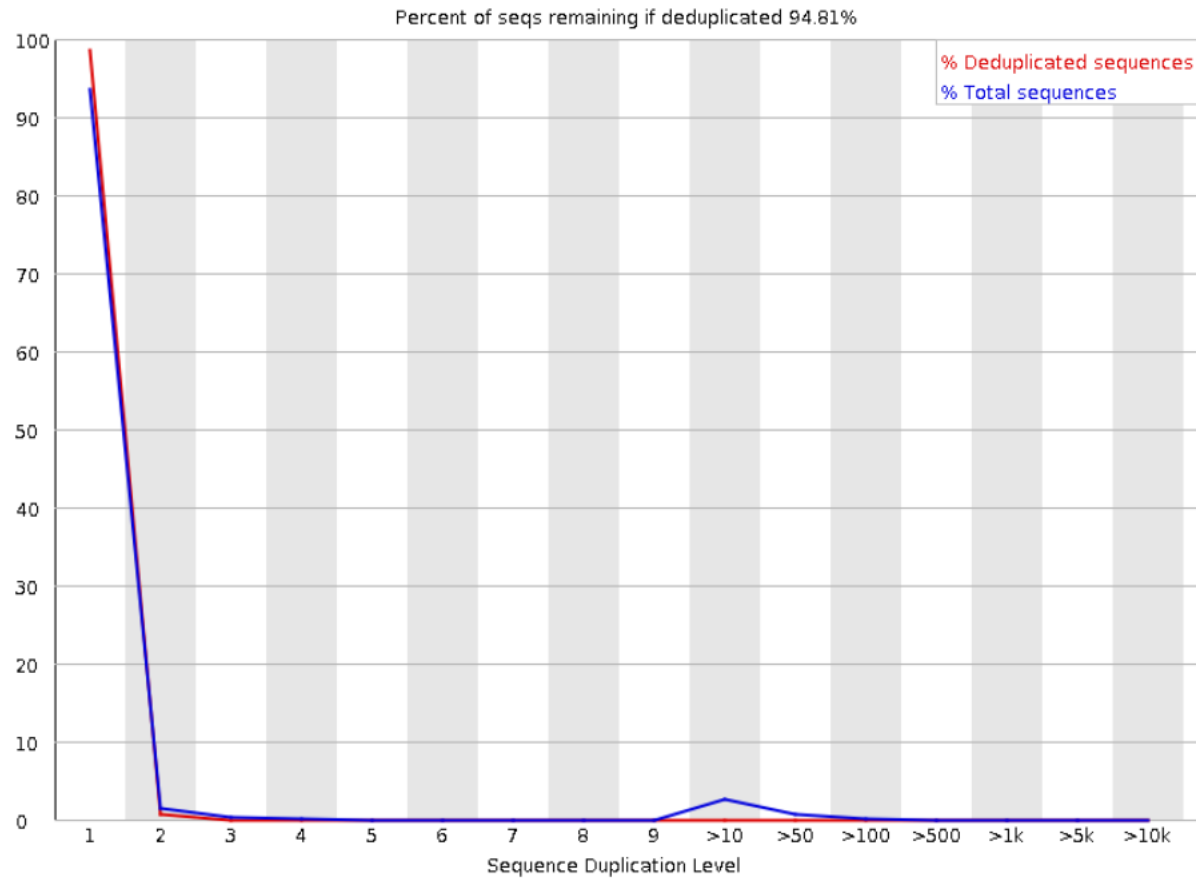


How can I Improve?:

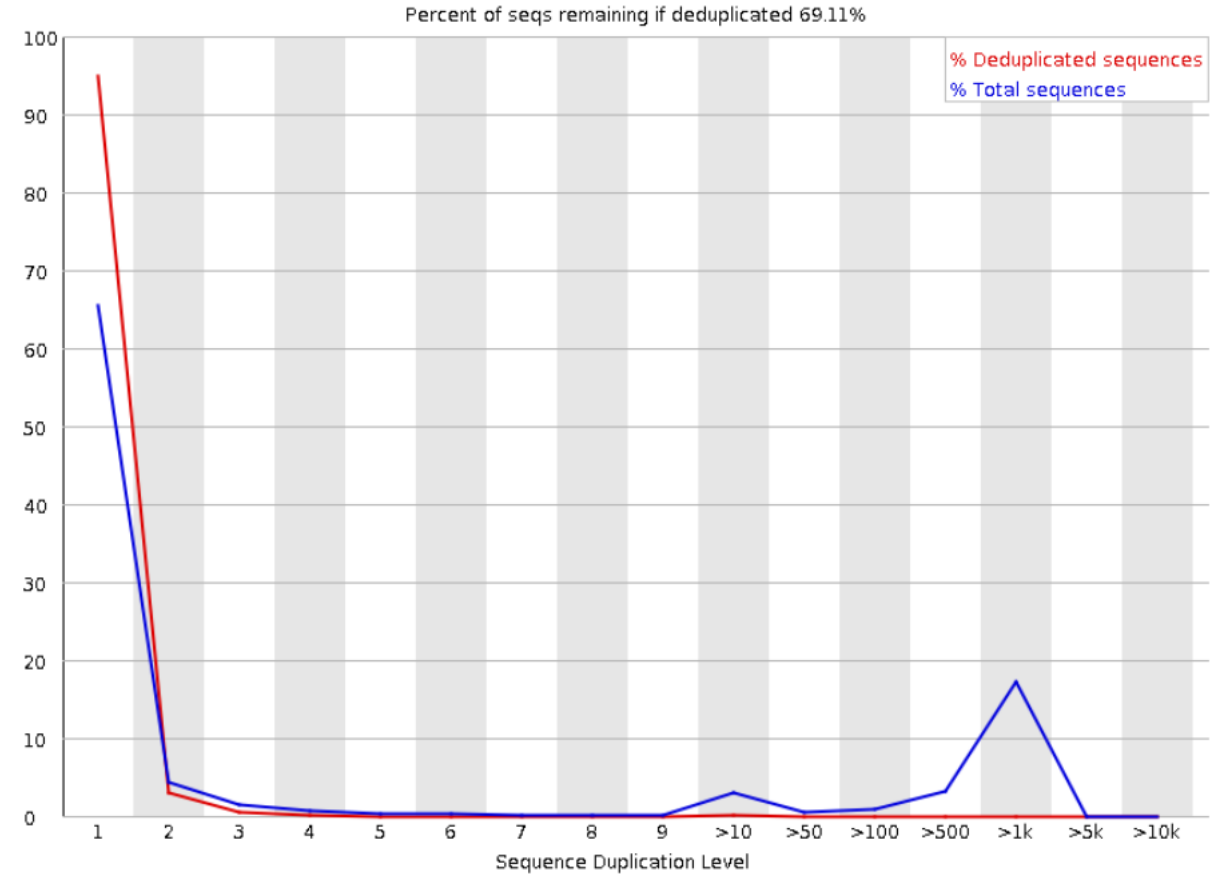
In genomics, two peaks might indicate contamination with another organism.
In transcriptomics, this value varies with the composition of the transcriptome

Read duplication

Good Quality Run



Bad Quality Run



How can I Improve?:

I don't do much at this point, I assess read duplication after alignment

Overrepresented sequences (more than 0,1% of the total)

Good Quality Run

Bad Quality Run

! Overrepresented sequences

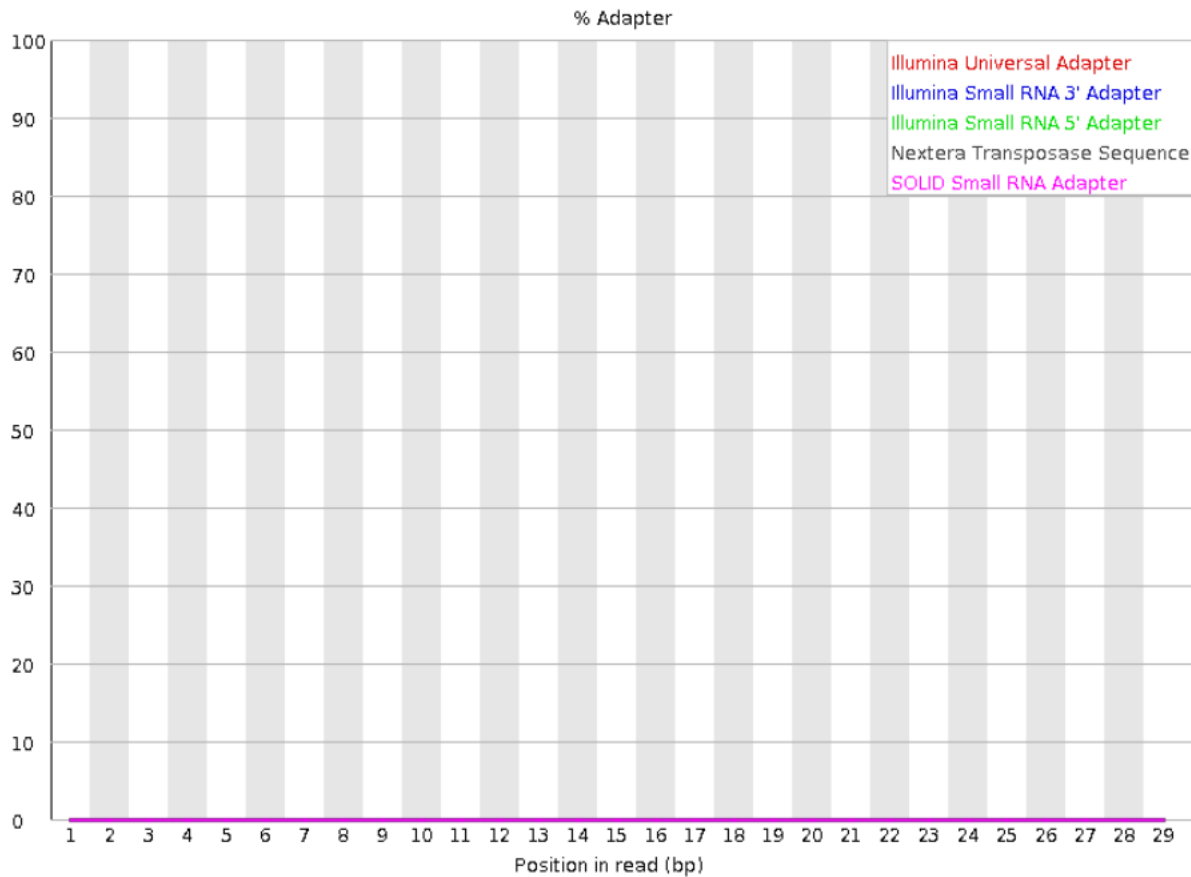
Sequence	Count	Percentage	Possible Source
AGAGTTTATCGCTTCCATGACGCAGAAGTTAACTTTC	2065	0.5224039181558763	No Hit
GATTGGCGTATCCAACCTGCAGAGTTTATCGCTTCCATG	2047	0.5178502762542754	No Hit
ATTGGCGTATCCAACCTGCAGAGTTTATCGCTTCCATGA	2014	0.5095019327680071	No Hit
CGATAAAATGATTGGCGTATCCAACCTGCAGAGTTTAT	1913	0.4839509420979134	No Hit
GTATCCAACCTGCAGAGTTTATCGCTTCCATGACGCAGA	1879	0.4753496185060066	No Hit
AAAAATGATTGGCGTATCCAACCTGCAGAGTTTATCGCT	1846	0.4670012750197325	No Hit
TGATTGGCGTATCCAACCTGCAGAGTTTATCGCTTCCAT	1841	0.46573637449150995	No Hit
AACCTGCAGAGTTTATCGCTTCCATGACGCAGAAGTTAA	1836	0.46447147396328753	No Hit
GATAAAATGATTGGCGTATCCAACCTGCAGAGTTTATC	1831	0.4632065734350651	No Hit
AAATGATTGGCGTATCCAACCTGCAGAGTTTATCGCTTC	1779	0.45005160794155147	No Hit
ATGATTGGCGTATCCAACCTGCAGAGTTTATCGCTTCCA	1779	0.45005160794155147	No Hit

How can I Improve?:

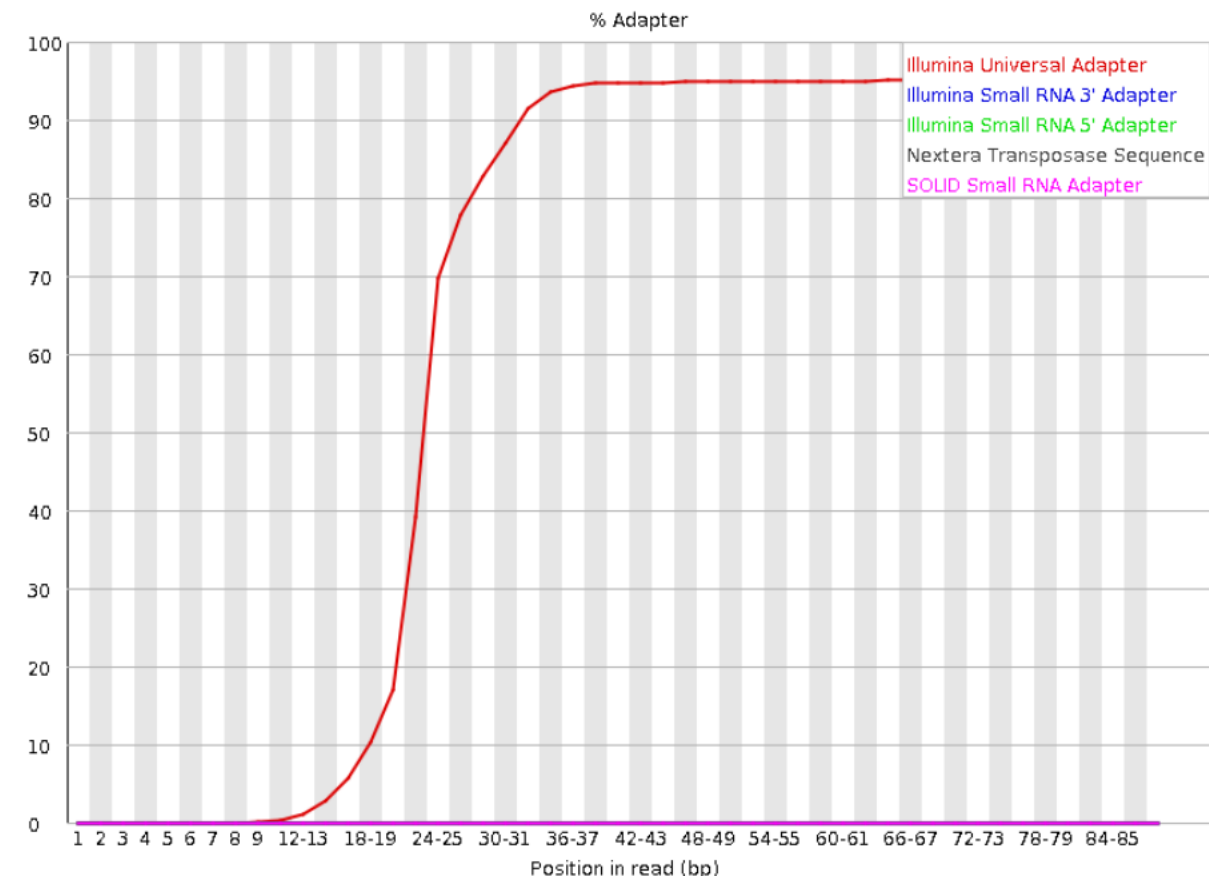
RNA-seq will tend to have overrepresented sequences coming from highly expressed genes
Just blast the sequences listed on the report to be such makes sense with your sample,
otherwise it might be contamination

Adapter Content

Good Quality Run



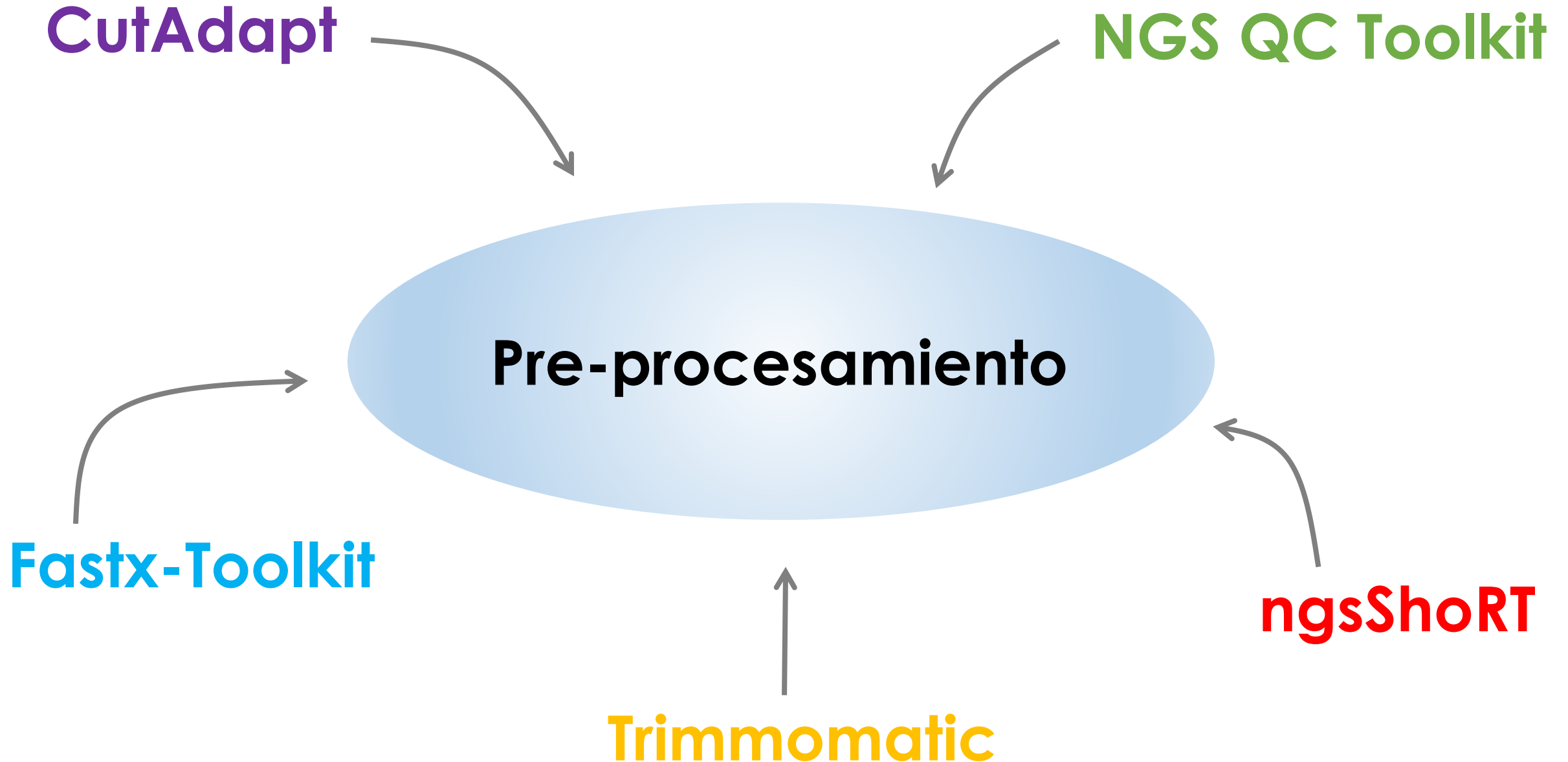
Bad Quality Run



How can I Improve?:

Clip adapter from your sequences using the pre-processing programs :D

Pre-processing programs will do the job to remove error and bad quality data



CutAdapt

<https://cutadapt.readthedocs.io/en/stable/index.html>

 **cutadapt**
stable

[Installation](#)
[User guide](#)
[Colospace reads](#)
[Algorithm details](#)
[Recipes \(FAQ\)](#)
[Ideas/To Do](#)
[Developing](#)
[Changes](#)

Docs » cutadapt

 [Edit on GitHub](#)

build passing pypi v1.17

cutadapt

Cutadapt finds and removes adapter sequences, primers, poly-A tails and other types of unwanted sequence from your high-throughput sequencing reads.

Cleaning your data in this way is often required: Reads from small-RNA sequencing contain the 3' sequencing adapter because the read is longer than the molecule that is sequenced. Amplicon reads start with a primer sequence. Poly-A tails are useful for pulling out RNA from your sample, but often you don't want them to be in your reads.

Cutadapt helps with these trimming tasks by finding the adapter or primer sequences in an error-tolerant way. It can also modify and filter reads in various ways. Adapter sequences can contain IUPAC wildcard characters. Also, paired-end reads and even colospace data is supported. If you want, you can also just demultiplex your input data, without removing adapter sequences at all.

Cutadapt comes with an extensive suite of automated tests and is available under the terms of the MIT license.

If you use cutadapt, please cite [DOI:10.14806/ej.17.1.200](https://doi.org/10.14806/ej.17.1.200) .

Opciones CutAdapt

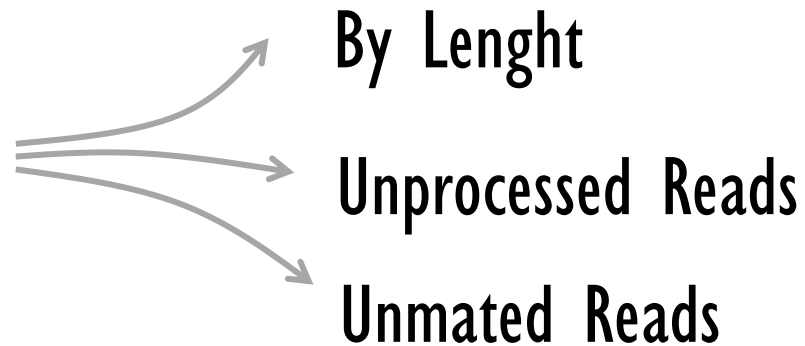
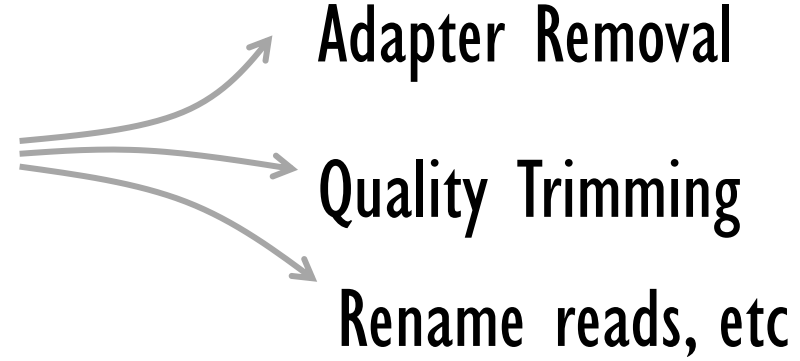
Step 1: Modificación de los reads



Step 2: Read's Filtering



Step 3: Output



Opciones CutAdapt

Understanding adapter type:

Adapter type	Command-line option
Regular 3' adapter	<code>-a ADAPTER</code>
Regular 5' adapter	<code>-g ADAPTER</code>
Non-internal 3' adapter	<code>-a ADAPTERX</code>
Non-internal 5' adapter	<code>-g XADAPTER</code>
Anchored 3' adapter	<code>-a ADAPTER\$</code>
Anchored 5' adapter	<code>-g ^ADAPTER</code>
5' or 3' (both possible)	<code>-b ADAPTER</code>
Linked adapter	<code>-a ADAPTER1...ADAPTER2</code>
Non-anchored linked adapter	<code>-g ADAPTER1...ADAPTER2</code>

Opciones CutAdapt

Processing Order:

1. Unconditional base removal with `--cut`
2. Quality trimming (`-q`)
3. Adapter trimming (`-a` , `-b` , `-g` and uppercase versions)
4. Read shortening (`--length`)
5. N-end trimming (`--trim-n`)
6. Length tag modification (`--length-tag`)
7. Read name suffix removal (`--strip-suffix`)
8. Addition of prefix and suffix to read name (`-x` / `--prefix` and `-y` / `--suffix`)
9. Double-encode the sequence (only colorspace)
10. Replace negative quality values with zero (zero capping, only colorspace)
11. Trim primer base (only colorspace)

Opciones CutAdapt

Step 2: Filtering

`--minimum-length LENGTH` or `-m LENGTH`

Discard processed reads that are shorter than LENGTH. Reads that are too short even before adapter removal are also discarded. Without this option, reads that have a length of zero (empty reads) are kept in the output.

`--too-short-output FILE`

Instead of discarding the reads that are too short according to `-m`, write them to *FILE* (in FASTA/FASTQ format).

`--maximum-length LENGTH` or `-M LENGTH`

Discard processed reads that are longer than LENGTH. Reads that are too long even before adapter removal are also discarded.

`--too-long-output FILE`

Instead of discarding reads that are too long (according to `-M`), write them to *FILE* (in FASTA/FASTQ format).

`--untrimmed-output FILE`

Write all reads without adapters to *FILE* (in FASTA/FASTQ format) instead of writing them to the regular output file.

`--discard-trimmed`

Discard reads in which an adapter was found.

`--discard-untrimmed`

Discard reads in which *no* adapter was found. This has the same effect as specifying

`--untrimmed-output /dev/null`.

It is recommended to perform trimming and filtering keeping paired-end reads synchronized.

Bonus - Trimmomatic trimming steps

ILLUMINACLIP: Cut adapter and other illumina-specific sequences from the read.

SLIDINGWINDOW: Perform a sliding window trimming, cutting once the average quality within the window falls below a threshold.

LEADING: Cut bases off the start of a read, if below a threshold quality

TRAILING: Cut bases off the end of a read, if below a threshold quality

CROP: Cut the read to a specified length

HEADCROP: Cut the specified number of bases from the start of the read

MINLEN: Drop the read if it is below a specified length

TOPHRED33: Convert quality scores to Phred-33

TOPHRED64: Convert quality scores to Phred-64

****Trimming occurs in the order which the steps are specified**

OUTPUT

For PE sequences, it will produce **4 outputs files**:

2 for “paired” outputs where both reads survived the processing

2 for 2 “unpaired” outputs where one one of the reads survived (the partner reads didn’t)